

11) Deployment Environments and Config

Last updated by | Mennell, Jacob | 19 May 2026 at 12:17 BST

Deployment, Environments, And Config

Purpose

This page explains how the solution runs locally, how services are composed, and how code is built and validated through the delivery pipeline.

Scope

The solution uses a container-first local development model and a pipeline-based build and scan model. Configuration is primarily environment-driven, with service-specific settings layered on top of shared base configuration.

Local Runtime Topology

The local environment is defined through the compose file and includes:

- PostgreSQL with pgvector
- RabbitMQ
- migrations container
- API service
- main worker
- ingestion worker
- frontend served through NGINX

The startup ordering indicates that the database must become healthy before migrations run, and worker services depend on migrations completing successfully.

Containerization Model

The backend services share a common backend Dockerfile with service-specific build arguments, while the frontend is built separately through its own Dockerfile.

This gives the solution a consistent containerization model across services while still allowing service-specific dependencies and runtime entry points.

Azure Deployment Footprint

The AUG Triage platform is deployed to Azure through a set of Azure Container Apps that cover frontend delivery, API handling, worker execution, scheduling, and migration tasks. These workloads are supported by platform services for messaging, identity, AI processing, and document analysis.

Azure Container Apps

Logical Service	Azure Resource	Role
Frontend	ca-claimsfe001-augtriage-fe	Serves the React application through NGINX
API	ca-claimsbe001-augtriage-api	Exposes backend endpoints, handles orchestration requests, and submits work to the async layer
Main Worker	ca-claimsbe001-augtriage-mw	Runs core extraction, validation, and AI-assisted workflow processing
Ingestion Worker	ca-claimsbe001-augtriage-iw	Handles document ingestion, parsing, chunking, embedding, and metadata enrichment
RabbitMQ	ca-claimsbe001-augtriage-rmq	Provides message brokering for Celery task delivery between services
Celery Beat	ca-claimsbe001-augtriage-cb	Triggers scheduled workflows and periodic background processing
Migration Runner	ca-claimsbe001-augtriage-miga	Applies schema migrations as part of deployment and environment lifecycle activities
Migration Job	ca-claimsbe001-augtriage-migj	Runs one-off or manually triggered migration tasks

Supporting Azure Services

The repository also indicates supporting services that sit alongside the deployed container apps:

- Azure Container Registry for image storage and deployment packaging
- Azure AD / Entra ID for authentication and role-aware access control
- Azure OpenAI or Azure AI Foundry-hosted model access for AI orchestration flows
- Azure Document Intelligence for document parsing and structured extraction support
- monitoring services such as Log Analytics or Application Insights for operational visibility
- PostgreSQL with pgvector where the environment hosts operational and retrieval data in Azure-managed infrastructure

This distinction is useful in external documentation because not every deployed platform component is itself a container app. The business services run in Azure Container Apps, while identity, storage, monitoring, and AI capabilities are provided by adjacent Azure services.

Configuration Model

Configuration is layered through shared base settings and service-specific settings classes.

Examples visible in the code include:

- API authentication toggles and role-elevation flags

- worker concurrency and retry controls
- retrieval defaults for the main worker
- ingestion chunk rebalancing settings for the ingestion worker
- RabbitMQ and database connection settings

Most environment differences are therefore expressed through configuration rather than code branching.

Environment Variables And Service Specialization

The main worker and ingestion worker each expose settings that reflect their responsibilities:

- the ingestion worker exposes parsing, retry, and chunking controls
- the main worker exposes retrieval defaults, concurrency controls, and source-coordinate behavior
- the API exposes authentication and integration-facing configuration

This separation makes it easier to tune individual services without changing unrelated runtime concerns.

Delivery Pipeline Summary

The build pipeline performs a combined validation and packaging flow that includes:

- Python version setup
- dependency installation for backend services and libraries
- unit test execution for API, main worker, ingestion worker, and aiutil
- coverage collection and publication
- frontend dependency installation and build
- SonarCloud preparation, analysis, and publication
- wheel and source-distribution builds for backend modules

The build process is therefore more than a packaging step; it is also a quality and analysis gate.

Delivery Pipelines And Environment Promotion

The solution uses separate Azure DevOps pipelines for build, deployment, and image promotion. This separation makes the release model easier to understand and control because validation, environment rollout, and production promotion are handled as distinct stages.

Pipeline Roles

Pipeline Type	Role
Build	Validates the codebase, runs tests and quality checks, and produces deployable artifacts and images
Deploy	Deploys the application components into the target environment using the relevant configuration
Promote	Promotes approved images from non-production to production without rebuilding them

This model helps preserve release discipline. Code is built and validated once, then deployed and promoted through controlled steps rather than being rebuilt independently for each environment.

Pipeline Links

Pipeline Type	Link	Notes
Build	Canopus.Apps.Claims.AugmentedTriage - Build	Main validation, test, analysis, and packaging pipeline
Deploy	Canopus.Apps.Claims.AugmentedTriage - Deploy	Deploys the solution into target environments
Promote	Canopus.Apps.Claims.AugmentedTriage - Promote	Promotes images from non-production to production

Access to these links depends on Azure DevOps permissions and is typically limited to internal delivery and support teams.

Environment Configuration Model

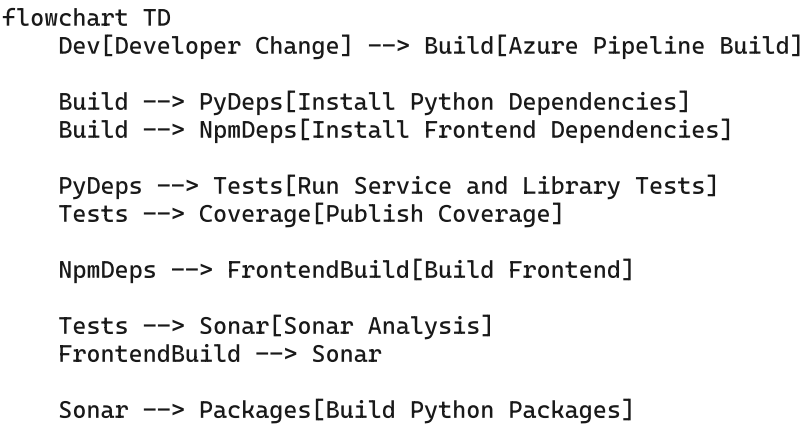
Environment-specific settings are managed separately from the shared deployment flow. The repository and deployment model indicate a set of environment-specific configuration groups for:

- DEV
- TEST
- UAT
- PROD

This allows the same deployment structure to be reused across environments while keeping environment values isolated. In practice, the pipeline logic stays broadly consistent, while credentials, endpoints, feature flags, and operational settings can vary by environment.

For an externally shared view of the platform, it is usually enough to describe these as environment-specific variable groups or configuration libraries rather than listing internal identifiers and administration links directly in the main documentation.

Deployment And Build Diagram



Design Decisions

- Local development is container-first, which reduces machine-specific setup drift.
- Migrations are treated as a first-class service dependency.
- Configuration is environment-driven and service-specific where appropriate.
- The pipeline validates backend services and shared libraries independently rather than treating the backend as one undifferentiated unit.

Operational Notes

- Runtime behavior can differ by environment based on configuration, especially for authentication and worker tuning.
- Pipeline results provide a consolidated signal for tests, coverage, and static analysis.
- Local startup depends on external configuration being available through environment files or environment variables.